

Python — SET (Part 1)

A set is an unordered collection of unique items.

Sets are mutable, but their individual elements must be hashable.

Sets use `{ }` or `set()`.

Duplicate values are automatically removed.

1. Creating a Set

```
s = {10, 20, 30, 40}
names = {"Jahanur", "Rahul", "Amit"}
mixed = {10, "Python", True}
empty = set()
empty_dict = {}    # dictionary
```

2. Duplicates Are Removed ☆

```
s = {1, 2, 2, 3, 3, 3}
print(s)    # {1, 2, 3}
```

3. Sets Have No Index

You cannot use `s[0]` or slicing on a set.

Use a loop or membership testing instead.

```
s = {"Python", "Java", "C++"}
# s[0]      × TypeError
for item in s:
    print(item)
```

4. Membership Testing

```
s = {10, 20, 30}
20 in s      # True
50 in s      # False
50 not in s   # True
```

Python — SET (Part 2)

5. Adding Elements

```
s = {1, 2, 3}
s.add(4)
print(s)    # {1, 2, 3, 4}
```

6. Adding Multiple Elements

```
s = {1, 2}
s.update([3, 4, 5])
print(s)    # {1, 2, 3, 4, 5}
```

7. Removing Elements

```
s = {10, 20, 30}
s.remove(20)    # error if absent
s.discard(50)   # safe if absent
s.pop()         # arbitrary item
s.clear()       # empty set
```

8. Important Set Methods

<code>add(x)</code>	— add one item
<code>update(x)</code>	— add many items
<code>remove(x)</code>	— remove; error if absent
<code>discard(x)</code>	— remove; no error
<code>pop()</code>	— remove arbitrary item
<code>clear()</code>	— empty set
<code>copy()</code>	— shallow copy

9. Length

```
s = {10, 20, 30, 40}
len(s)    # 4
```

Python — SET OPERATIONS (Part 3)

10. Union — Combine Sets ☆

```
A = {1, 2, 3}
B = {3, 4, 5}
A | B          # {1,2,3,4,5}
A.union(B)     # same
```

11. Intersection — Common ☆

```
A = {1, 2, 3}
B = {2, 3, 4}
A & B          # {2,3}
A.intersection(B)
```

12. Difference ☆

A - B gives elements in A but not B.

```
A = {1, 2, 3}
B = {2, 3, 4}
A - B          # {1}
B - A          # {4}
```

13. Symmetric Difference

Elements in either set, but not in both.

```
A = {1, 2, 3}
B = {2, 3, 4}
A ^ B          # {1,4}
A.symmetric_difference(B)
```

14. Memory Trick

UNION → everything

INTERSECTION → common

DIFFERENCE → left-only

SYMMETRIC → not common

Python — SET (Part 4)

15. Subset & Superset ☆

```
A = {1, 2}
B = {1, 2, 3, 4}
A.issubset(B)      # True
B.issuperset(A)    # True
```

16. Disjoint Sets

```
A = {1, 2}
B = {3, 4}
A.isdisjoint(B)    # True
```

17. Set vs List vs Tuple

LIST → ordered, mutable, duplicates allowed

TUPLE → ordered, immutable, duplicates allowed

SET → unordered, mutable, unique elements

18. When to Use a Set?

- Remove duplicates.
- Fast membership checking.
- Find common/unique values.
- Union, intersection, difference operations.

19. DSA Connection ?

Sets are excellent for membership and duplicate-detection problems.

Average-case membership lookup is $O(1)$.

Use a set to detect duplicate numbers in an array.

20. Practice Programs

1. Remove duplicates from a list using a set.
2. Find common elements of two lists.
3. Find A-only elements.
4. Find union and intersection.
5. Check subset/superset.
6. Detect duplicate numbers.
7. Count unique words.
8. Find symmetric difference.

21. Quick Revision

Set = unordered + unique elements.

Empty set = `set()`, not `{}`.

No indexing or slicing.

`add()` one; `update()` many.

`remove()` may error; `discard()` does not.

`|` union, `&` intersection, `-` difference, `^` symmetric difference.

Practice every concept with code — understanding comes from doing.